

JavaScript Array Traversal using **for** Loop

The **for** **loop** is the most commonly used loop for traversing (iterating through) arrays in JavaScript.

It allows you to access every element one by one using its **index**.

Table of Contents

- What is a for Loop?
- Why Use a for Loop?
- Syntax
- How a for Loop Works
- Array Index
- Basic Examples
- Multiple Examples
- Real Project Examples
- Best Practices
- Common Mistakes
- Interview Questions
- Summary Table

What is a for Loop?

Definition

A **for** loop repeatedly executes a block of code until a specified condition becomes **false** .

When working with arrays, it is mainly used to access every element using its index.

Why Use a for Loop?

We use a **for** loop to:

- Traverse an array
- Print array elements

- Calculate totals
- Search data
- Update array values
- Process API data

Syntax

```
for(initialization; condition; update){  
  
    // code  
  
}
```

Array Syntax

```
for(let i = 0; i < array.length; i++){  
  
    console.log(array[i]);  
  
}
```

Understanding the Loop

```
const fruits = [  
  
    "Apple",  
  
    "Mango",  
  
    "Orange"  
  
];
```

Iteration	i	fruits[i]
1	0	Apple
2	1	Mango
3	2	Orange

When **i** becomes **3** , the condition

```
i < fruits.length
```

becomes

```
3 < 3
```

which is **false** , so the loop stops.

Example 1 (Print All Elements)

```
const fruits = [
  "Apple",
  "Mango",
  "Orange"
];

for(let i = 0; i < fruits.length; i++){
  console.log(fruits[i]);
}
```

Output

Apple
Mango
Orange

Example 2 (Numbers)

```
const numbers = [  
  10,  
  20,  
  30,  
  40  
];  
  
for(let i = 0; i < numbers.length; i++){  
  console.log(numbers[i]);  
}
```

Output

```
10  
20  
30  
40
```

Example 3 (Using Index)

```
const colors = [
```

```
    "Red",  
  
    "Green",  
  
    "Blue"  
  
];  
  
for(let i = 0; i < colors.length; i++){  
    console.log(i, colors[i]);  
}
```

Output

```
0 Red  
  
1 Green  
  
2 Blue
```

Example 4 (Sum of Array)

```
const numbers = [  
  
    10,  
  
    20,  
  
    30,  
  
    40  
  
];  
  
let sum = 0;  
  
for(let i = 0; i < numbers.length; i++){  
    sum += numbers[i];  
}
```

```
}  
  
console.log(sum);
```

Output

```
100
```

Example 5 (Find Largest Number)

```
const numbers = [  
    20,  
    60,  
    15,  
    80,  
    40  
];  
  
let largest = numbers[0];  
  
for(let i = 1; i < numbers.length; i++){  
    if(numbers[i] > largest){  
        largest = numbers[i];  
    }  
}  
  
console.log(largest);
```

Output

```
80
```

Example 6 (Print Even Numbers)

```
const numbers = [  
  2,  
  5,  
  8,  
  11,  
  20  
];  
  
for(let i = 0; i < numbers.length; i++){  
  if(numbers[i] % 2 === 0){  
    console.log(numbers[i]);  
  }  
}
```

Output

```
2  
8  
20
```

Example 7 (Update Every Element)

```
const prices = [  
  100,  
  200,  
  300  
];  
  
for(let i = 0; i < prices.length; i++){  
  prices[i] = prices[i] + 50;  
}  
  
console.log(prices);
```

Output

```
[150,250,350]
```

Example 8 (Reverse Traversal)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange"  
];  
  
for(let i = fruits.length - 1; i >= 0; i--){
```



```
    console.log(fruits[i]);  
  
}
```

Output

```
Orange  
Mango  
Apple
```

Example 9 (Search an Element)

```
const students = [  
    "Rahim",  
    "Karim",  
    "Rokon"  
];  
  
let found = false;  
  
for(let i = 0; i < students.length; i++){  
    if(students[i] === "Rokon"){  
        found = true;  
        break;  
    }  
}  
  
console.log(found);
```

Output

```
true
```

Example 10 (Count Elements)

```
const fruits = [  
  "Apple",  
  "Mango",  
  "Orange",  
  "Banana"  
];  
  
let count = 0;  
  
for(let i = 0; i < fruits.length; i++){  
  count++;  
}  
  
console.log(count);
```

Output

```
4
```

Real Project Example 1

Shopping Cart

```
const cart = [  
  "Laptop",  
  "Mouse",  
  "Keyboard"  
];  
  
for(let i = 0; i < cart.length; i++){  
  console.log(cart[i]);  
}
```

Real Project Example 2

Student Result

```
const marks = [  
  80,  
  75,  
  95  
];  
  
let total = 0;  
  
for(let i = 0; i < marks.length; i++){  
  total += marks[i];  
}  
  
console.log(total);
```

Output

250

Real Project Example 3

Product Price

```
const prices = [  
  100,  
  200,  
  300  
];  
  
for(let i = 0; i < prices.length; i++){  
  console.log("Price:", prices[i]);  
}
```

Real Project Example 4

Attendance List

```
const students = [  
  "Rahim",  
  "Karim",  
  "Rokon"
```

```
];  
  
for(let i = 0; i < students.length; i++){  
    console.log("Present:", students[i]);  
}
```

Best Practices

Use `array.length` instead of writing fixed numbers.

```
for(let i = 0; i < array.length; i++)
```

Use meaningful variable names.

```
students  
  
products  
  
prices  
  
marks
```

Keep loop logic simple and readable.

Common Mistakes

Mistake 1

Using `<=`

Wrong

```
for(let i = 0; i <= array.length; i++)
```

Correct

```
for(let i = 0; i < array.length; i++)
```

Mistake 2

Forgetting `i++`

Wrong

```
for(let i = 0; i < array.length){  
  }  
}
```

This creates an infinite loop.

Mistake 3

Starting from the wrong index.

Wrong

```
for(let i = 1; i < array.length; i++)
```

This skips the first element.

Mistake 4

Accessing an invalid index.

```
array[array.length]
```

returns

```
undefined
```

Interview Questions

What is the purpose of a **for** loop?

It repeats a block of code until a condition becomes false.

Why is **array.length** used?

To avoid going beyond the last element.

Which index does an array start from?

```
0
```

How do you print an array in reverse order?

```
for(let i = array.length - 1; i >= 0; i--){  
  console.log(array[i]);  
}
```

How do you find the sum of all elements?

Use a variable and add each element inside the loop.

Summary Table

Feature	Description
Loop	for
Purpose	Traverse an array
Starts From	0
Ends At	array.length - 1
Access Element	array[i]
Changes Array	Optional
Common Uses	Print, Search, Sum, Update, Traverse

Final Notes

The **for loop** is the foundation of array traversal in JavaScript. Before learning advanced methods like **for...of** , **map()** , **filter()** , and **reduce()** , you should be comfortable using the **for** loop because it gives you complete control over the array index and iteration process.